

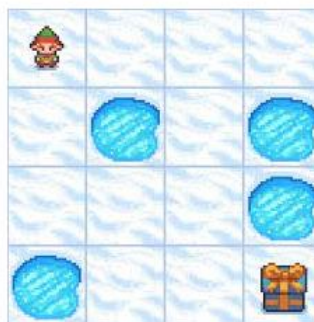
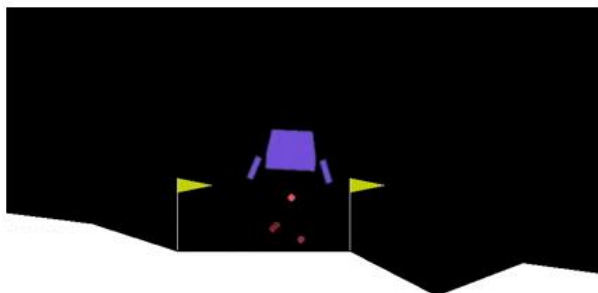
STEROWANIE I LOGIKA ROZMYTA

+ KRÓTKIE ZADANIE NA REGUŁY ASOCJACYJNE

📖 ZADANIE 1: WIRTUALNE ŚRODOWISKA W GYM

Na wykładzie były pokazywane biblioteki Gym i Gymnasium do symulacji pewnych procesów (lub grania) w wirtualnym środowisku.: <https://gymnasium.farama.org/index.html>.

- a) Uruchom pliki pythonowe z wykładu. Sprawdź, czy gry LunarLander i FrozenLake działają. Pamiętaj o zainstalowaniu odpowiednich paczek i ewentualnych dodatkowych czynnościach (instalacja swig -> patrz wykład).



- b) Uruchom dodatkowe gry. Najlepiej po jednej z pięciu kategorii (Classic Control, Box2D, ToyText, MuJoCo, Atari). Uwaga! Każda kategoria może wymagać instalacji dodatkowych bibliotek.
- c) Przeglądając dokumentację, zidentyfikuj po jednej grze z następujących kategorii:
- Stan gry i zestaw akcji są dyskretne (tzn. jest to skończony zestaw).
 - Stan gry jest ciągły (nieskończony, liczby zmiennoprzecinkowe), ale zestaw akcji jest dyskretny.
 - Stan gry i zestaw akcji jest ciągły.
- Uruchom te gry i dopisz w komentarzy w kodzie, który z typów to jest.
- d) Dla dwóch wybranych gier, podmień losowe próbki akcji, na zestaw akcji stworzony przez Ciebie, który będzie dobrze działał („dobrze” tzn. osiągniemy sukces, albo przynajmniej zbliżymy się do osiągnięcia sukcesu).

📖 ZADANIE 2: KONTROLER ROZMYTY I NAPIWKI

Należę Użyjemy paczki simpful, by stworzyć prosty kontroler rozmyty do obliczania napiwków (przykład był na wykładzie).

Poczytaj o paczce simpful np. tutaj:

- <https://pypi.org/project/simpful/>
- https://pythonhosted.org/scikit-fuzzy/auto_examples/plot_tipping_problem_newapi.html
- https://www.researchgate.net/publication/346395808_Simpful_A_User-Friendly_Python_Library_for_Fuzzy_Logic

Zwróć uwagę na wstawki kodu i sposób tworzenia kontrolerów (zmienne, reguły, wyostrzanie). Zainstaluj paczkę i skopiuj z wybranej strony kod tworzący system do dawania napiwków (3 zmienne lingwistyczne, 3 reguły). Wyświetl wykresy zmiennych lingwistycznych. Przetestuj działanie kontrolera. Daj kilka danych (liczby dla jedzenia i obsługi) i wyświetl jaki napiwek (0-30%) proponuje system dla tych inputów.

ZADANIE 3: KONTROLER ROZMYTY W STEROWANIU

Wykorzystaj paczkę `simful` (lub inne rozwiązanie) do stworzenia kontrolera rozmytego dla wybranej gry z Gym (tutaj najlepiej sprawdzi się gra ze stanami i akcjami ciągłymi/zmiennoprzecinkowymi) np.

- LunarLander (wersja continuous)
- Bipedal Walker
- Pendulum (łatwiejsze niż LunarLander)

Rozwiązanie powinno się składać z następujących kroków:

- a) Zdefiniowanie sensownych zmiennych lingwistycznych dla każdej zmiennej. Wyświetl wykresy.
- b) Zdefiniowanie zestawu sensownych reguł wnioskowania rozmytego. Zastanów się czy lepszym operatorem będą „or” czy „and”.
- c) Odpalenie systemu i obliczanie wyostrzonych zmiennych sterujących (consequents). Zmienne powinny być obliczane w każdej klatce symulacji. W każdej klatce animacji fuzzy controller oblicza co trzeba zrobić i wykonuje tę akcję.

Postaraj się, by system działał w miarę dobrze (nie musi działać idealnie). W razie czego zmodyfikuj zmienne lingwistyczne lub reguły rozmyte.

ZADANIE 4: TITANIC I REGUŁY ASOCJACYJNE

Ponad 100 lat temu zatonął Titanic. Na pokładzie znajdowały się tysiące ludzi. Niektórzy przeżyli, inni nie. Czy były jakieś zależności pomiędzy płcią, zamożnością i przeżywalnością? Twoim zadaniem jest znalezienie reguł asocjacyjnych, które odpowiedzą na te pytania. Wczytaj plik `titanic.csv` ze strony lub z Internetu i upewnij się, że ma tylko 4 kolumny: "Class", "Sex", "Age", "Survived".

Rozwiązanie powinno zawierać:

1. Uruchomienie algorytmu Apriori na bazie danych.
2. Wyszukanie reguł o sensownych parametrach (minimum 0.005, i ufności 0.8) i posortowanie ich wg ufności.
3. Wyszukanie najciekawszych reguł. W szczególności wskazujących, kto przeżywał a kto nie. Można tutaj próbować odfiltrować niepotrzebne reguły.
4. Zobrazowanie reguł na pomocniczych wykresach i interpretacja ich. Zaprezentuj przynajmniej jeden.

Skorzystaj z jednego z samouczków:

<https://medium.com/analytics-vidhya/association-analysis-in-python-2b955d0180c>

<https://www.codespeedy.com/apriori-algorithm-in-python/>

<https://pbpython.com/market-basket-analysis.html>

<https://towardsdatascience.com/mba-for-breakfast-4c18164ef82b>

